H2H Harness-to-Harness: um protocolo para delegação local entre harnesses de agentes de codificação

Sumário

Sumário 1
Resumo 3
Abstract 3
1. Introdução 4
2. Trabalhos relacionados 5
2.1 Harness engineering 5
2.2 Model Context Protocol 5
2.3 Agent2Agent Protocol 5
2.4 Agent Client Protocol 5
2.5 Orquestração externa multi-harness 6
2.6 Lacuna investigada 6
3. Terminologia e modelo de delegação 6
4. Metodologia 7
4.1 Desenho da investigação 7
4.2 Unidade de análise 7
4.3 Critério de sucesso 8
4.4 Natureza do estudo 8
5. Proposta de protocolo H2H v0.1 8
5.1 Escopo 8
5.2 Manifesto de harness 8
5.3 Descritor de tarefa 9
5.4 Vocabulário comum de autonomia 9
5.5 Guarda contra recursão 10

5.6 Soberania de permissão do delegado 10
5.7 Timeout, saída e retomada 10
5.8 Uso de ACP como transporte 10
6. Estudos de caso 11
6.1 Fechamento de lacuna categórica: geração de imagens 11
6.2 Delegação de julgamento: trabalho editorial e analítico 12
6.3 Triangulação de um defeito real 12
6.4 Achados transversais 13
7. Discussão 14
7.1 Três modos de valor 14
7.2 Relação com protocolos existentes 14
7.3 Segurança e responsabilidade 14
7.4 Reprodutibilidade prática 15
8. Limitações e ameaças à validade 15
9. Trabalhos futuros 15
10. Conclusão 16
Referências 16

Autores: Jair Lima; Jackson Mallmann
Versão do artigo: 1.1
Data: 31 de julho de 2026

Sobre os autores: Jair Lima é programador desde 1993, com atuação independente em pesquisa e desenvolvimento de ferramentas e protocolos. É graduado em Redes de Computadores pelo Instituto Federal Catarinense (2014), especialista em Governança de TI pelo Centro Universitário Leonardo da Vinci (2015) e em Tecnologias Google for Education pela Universidade LaSalle (2019), e possui o certificado Chronicle Security Operations Platform – Deep Dive (Google Cloud Skills Boost, 2024), além das certificações CCNA (Cisco), Windows Server (Microsoft), Educator (Google) e Expert (Adobe). Contato: jairslima@gmail.com.

Dr. Jackson Mallmann é professor efetivo do Instituto Federal Catarinense (IFC), onde atua desde 2012 no ensino, na pesquisa e no desenvolvimento tecnológico nas áreas de Redes de Computadores, Segurança Computacional, Cibersegurança e Inteligência Artificial. É doutor em Informática pela Pontifícia Universidade Católica do Paraná (2021), mestre em Informática pela mesma instituição (2011) e bacharel em Ciência da Computação pela Universidade do Estado de Santa Catarina (2004). Possui formação complementar em CyberOps Associate e Network Security pela Cisco, Google Cloud Computing Foundations, segurança e identidade em ambientes de nuvem, computação forense, proteção de dados, aprendizagem de máquina e deep learning. É autor de artigos científicos, livros e capítulos técnicos, coordena pesquisas sobre sistemas de detecção de intrusões e identificação de ataques de ransomware por inteligência artificial e possui patente e

registros de programas de computador na área de detecção automatizada de conteúdo em vídeo. Currículo Lattes: http://lattes.cnpq.br/4046837503511326.

Resumo

Os harnesses de agentes de codificação combinam modelos de linguagem, ferramentas, memória, políticas de permissão e mecanismos de verificação em ambientes operacionais completos. Embora protocolos como MCP, A2A e ACP tenham avançado a interoperabilidade entre aplicações, ferramentas e agentes, permanece pouco sistematizado um cenário específico: um harness de codificação decidir, durante a execução de uma tarefa, invocar outro harness completo como subprocesso local, delegar-lhe uma parte do trabalho e receber de volta resultados e artefatos verificáveis, sem depender de um orquestrador externo. Este artigo denomina esse padrão **H2H, Harness-to-Harness**, e apresenta uma proposta mínima de protocolo derivada de uma investigação exploratória intensiva, realizada ao longo de poucos dias em ambiente Windows 11. O corpus compreendeu oito ferramentas ou superfícies de execução e seis casos empíricos principais, dos quais cinco corresponderam a tarefas de produção e um a uma prova sintética de capacidade. Os resultados indicam três modos de valor recorrentes: fechamento de lacunas categóricas de capacidade, obtenção de julgamento independente e triangulação de diagnósticos em problemas difíceis. A investigação também identificou riscos operacionais: recursão entre harnesses, elevação indevida de autonomia, ausência de timeout efetivo, perda de diagnóstico por truncamento de saída, dependências externas não declaradas e deriva de interfaces de linha de comando. Como resposta, o H2H v0.1 propõe manifesto de capacidades, descritor de tarefa, vocabulário comum de autonomia, profundidade máxima de delegação, soberania de permissão do harness delegado, captura integral de saída em falhas e verificação do estado dos artefatos antes da retomada. O trabalho deve ser interpretado como relato técnico exploratório e proposta de protocolo, não como benchmark comparativo de desempenho.

Palavras-chave: agentes de codificação; harness engineering; delegação de agentes; interoperabilidade; automação local; sistemas multiagentes.

Abstract

Coding-agent harnesses combine language models, tools, memory, permission policies, and verification mechanisms into complete operational environments. Although protocols such as MCP, A2A, and ACP have advanced interoperability among applications, tools, and agents, one specific scenario remains insufficiently systematized: a coding harness autonomously invoking another complete harness as a local subprocess, delegating part of a task, and receiving verifiable outputs and artifacts without relying on an external orchestrator. This article names that pattern **H2H, Harness-to-Harness**, and presents a minimal protocol proposal derived from an intensive exploratory investigation conducted over a few days on Windows 11. The corpus included eight tools or execution surfaces and six main empirical cases, five involving production tasks and one serving as a synthetic capability test. The results indicate three recurring value modes: closing categorical capability gaps, obtaining independent judgment, and triangulating diagnoses in difficult problems. The investigation also identified operational risks, including recursive invocation, excessive autonomy, unenforced timeouts, diagnostic loss caused by truncated output, undeclared external dependencies, and command-line interface drift. In response, H2H v0.1 proposes capability manifests, task descriptors, a shared autonomy vocabulary, a maximum delegation depth,

permission sovereignty for the delegated harness, complete failure-output capture, and artifact-state verification before task takeover. This work should be interpreted as an exploratory technical report and protocol proposal rather than a controlled performance benchmark.

Keywords: coding agents; harness engineering; agent delegation; interoperability; local automation; multi-agent systems.

1. Introdução

O desempenho de um agente de codificação não depende apenas do modelo de linguagem. Ele resulta da interação entre modelo, ferramentas, memória, contexto, permissões, observabilidade, mecanismos de execução e critérios de conclusão. Esse conjunto operacional é frequentemente descrito como *harness* (ZHONG; ZHU, 2026; CHEN et al., 2026).

Na prática, diferentes harnesses oferecem combinações distintas de modelos, ferramentas nativas, políticas de segurança, integrações e modos não interativos. Mesmo quando um produto permite selecionar mais de um modelo ou provedor, as ferramentas, as skills, a memória, a interface de execução e as políticas de autorização continuam pertencendo ao harness. Portanto, substituir apenas o modelo não equivale a transferir a capacidade operacional completa de um ambiente para outro.

Vários harnesses já oferecem execução programática por linha de comando, com entrada por argumento ou `stdin` e saída textual ou estruturada (ANTHROPIC, 2026; GOOGLE, 2026; GITHUB, 2026). Isso torna tecnicamente possível que um processo invoque outro e trate o resultado como parte de um fluxo maior. Contudo, a simples existência de comandos headless não resolve questões de descoberta de capacidades, autonomia, recursão, timeout, integridade de artefatos, dependências externas e atribuição de responsabilidade.

Este artigo propõe o conceito **H2H, Harness-to-Harness**, definido como a delegação local em que um harness chamador, por decisão própria ou por política previamente autorizada, invoca outro harness completo como subprocesso, transmite um descritor de tarefa, recebe saída e artefatos e conserva a responsabilidade pela coordenação da tarefa principal.

A investigação foi orientada por quatro perguntas:

1. Em que situações a delegação entre harnesses produz valor que um único harness não oferece com a mesma facilidade?
2. Quais falhas operacionais aparecem de forma recorrente quando um harness invoca outro localmente?
3. Quais elementos mínimos devem compor um protocolo H2H seguro e verificável?
4. Como o H2H se relaciona com protocolos existentes sem duplicar seus objetivos?

As principais contribuições são: a definição do padrão; sua distinção em relação a MCP, A2A, ACP e planos de controle externos; uma especificação mínima derivada de observações empíricas; e a organização dos resultados em três modos de valor, capacidade, julgamento e triangulação diagnóstica.

2. Trabalhos relacionados

2.1 Harness engineering

A literatura recente de harness engineering investiga como memória, ferramentas, observabilidade, controle de fluxo, avaliação, permissões e estruturas de contexto influenciam o desempenho de agentes (ZHONG; ZHU, 2026; WANG et al., 2026; CHEN et al., 2026; PAN et al., 2026; SAYED ALI et al., 2026). Esses trabalhos tratam o harness como objeto técnico próprio, e não como simples invólucro do modelo.

O H2H compartilha essa premissa, mas desloca o foco. Em vez de examinar apenas a arquitetura interna de um harness, investiga a composição operacional entre harnesses independentes e completos.

2.2 Model Context Protocol

O Model Context Protocol, MCP, padroniza a conexão entre aplicações hospedeiras e servidores que expõem ferramentas, recursos e prompts. Sua arquitetura favorece a composição de capacidades especializadas e pode operar por `stdio` local ou por transporte HTTP (MODEL CONTEXT PROTOCOL, 2025).

Um servidor MCP, entretanto, não precisa ser um harness de codificação completo. Em geral, ele expõe capacidades focadas sob o controle de uma aplicação hospedeira. O H2H trata de outra unidade de composição: um ambiente agente completo, com modelo, ferramentas, memória, políticas e ciclo de execução próprios, delegado como caixa-preta operacional.

2.3 Agent2Agent Protocol

O Agent2Agent Protocol, A2A, padroniza a comunicação entre sistemas agentes independentes e potencialmente opacos. O protocolo define descoberta por Agent Cards, mensagens, tarefas, artefatos e transportes de rede (A2A PROJECT, 2026). Seu escopo é suficientemente amplo para representar colaboração entre agentes e pode, em princípio, ser usado em uma máquina local por meio de um endpoint de rede.

A diferença central não é que A2A seja incapaz de descrever delegação, mas que o H2H busca uma solução menor e orientada a subprocessos locais, sem exigir que cada harness seja publicado como serviço. Além disso, o H2H torna normativas questões que surgiram na execução de CLIs locais, como profundidade de delegação, mapeamento de flags de autonomia, captura de `stdout` e `stderr`, diretório de trabalho, timeout imposto pelo chamador e verificação de alterações parciais.

2.4 Agent Client Protocol

O Agent Client Protocol, ACP, padroniza sessões entre clientes e agentes de codificação. Em seu modo local, o cliente inicializa o agente como subprocesso e a comunicação ocorre por JSON-RPC sobre `stdin` e `stdout`; o ecossistema também trabalha na estabilização de transportes remotos (AGENT CLIENT PROTOCOL, 2026a; 2026b).

ACP é especialmente próximo do H2H no nível de transporte. Ele já fornece inicialização, sessões, streaming e solicitações de permissão. Contudo, ACP não define, por si só, a política pela qual um harness decide delegar a outro, nem estabelece o conjunto de salvaguardas específicas proposto

neste trabalho. Assim, uma implementação H2H pode usar ACP quando o harness delegado o suportar e recorrer à CLI nativa como fallback.

2.5 Orquestração externa multi-harness

Planos de controle como o Oz, da Warp, executam e coordenam múltiplos harnesses a partir de uma camada externa, com observabilidade, governança e paralelismo (WARP, 2026). Esse modelo é valioso, mas difere do H2H quanto ao ponto de decisão. No plano de controle, um sistema superior coordena os agentes; no H2H, a decisão de delegar pode surgir dentro do próprio harness chamador durante a tarefa.

2.6 Lacuna investigada

No corpus de literatura e especificações consultado, não foi identificado um protocolo mínimo dedicado ao caso em que um harness de codificação, executando localmente, invoca outro harness completo como subprocesso, preserva a coordenação da tarefa principal e aplica salvaguardas normativas de recursão, autonomia, timeout e integridade de artefatos.

Essa formulação é deliberadamente restrita. O H2H não pretende substituir protocolos gerais de comunicação entre agentes, nem sistemas externos de orquestração. Ele busca padronizar uma prática local que já pode ser realizada por scripts, mas permanece fragmentada e dependente de convenções particulares.

Trabalhos recentes aproximam-se do problema sob perspectivas complementares. Recursive Agent Harnesses investiga a geração de processos que iniciam outros harnesses agentes, com ênfase em recursão e paralelismo. SentinelAgent formaliza cadeias verificáveis de delegação e preservação de políticas em sistemas multiagentes. Code as Agent Harness examina o código como infraestrutura executável para coordenação e verificação de agentes. O H2H distingue-se desses trabalhos por propor um contrato mínimo para delegação local, síncrona e limitada à profundidade 1 entre harnesses completos, utilizando CLI nativa ou ACP e preservando a soberania das políticas do delegado.

3. Terminologia e modelo de delegação

Para fins deste artigo, adotam-se as seguintes definições:

Termo	Definição
Harness de codificação	Ambiente operacional que combina um ou mais modelos, ferramentas, memória, contexto, permissões, execução e verificação de tarefas de software ou produção técnica.
Harness chamador	Harness que mantém a responsabilidade pela tarefa principal e inicia a delegação.
Harness delegado	Harness invocado para executar uma subtarefa delimitada.
Manifesto de harness	Documento declarativo com forma de invocação, capacidades, níveis de autonomia, dependências e restrições conhecidas.
Descritor de tarefa	Estrutura transmitida ao harness delegado com objetivo, contexto, diretório de trabalho, autonomia, timeout, formato de saída e histórico relevante.

Profundidade de delegação	Número de transições H2H consecutivas dentro da mesma tarefa raiz.
Artefato-alvo	Arquivo, diretório, branch, imagem, relatório ou outro objeto que pode ser criado ou modificado pela tarefa delegada.

O modelo mínimo possui quatro etapas:

1. O chamador identifica uma subtarefa e seleciona um delegado compatível.
2. O chamador consulta o manifesto e prepara o descritor da tarefa.
3. O delegado executa a subtarefa sob suas próprias políticas de segurança.
4. O chamador valida a saída, verifica os artefatos e decide aceitar, rejeitar, complementar ou delegar novamente a outro harness, respeitada a profundidade máxima.

O harness delegado é tratado como caixa-preta quanto ao raciocínio interno, mas não quanto aos efeitos observáveis. Comandos executados, arquivos modificados, saídas, códigos de retorno e verificações devem permanecer auditáveis sempre que a ferramenta os disponibilizar.

4. Metodologia

4.1 Desenho da investigação

O estudo adotou uma abordagem exploratória e indutiva. Em vez de definir antecipadamente uma especificação completa, foram realizados testes de delegação e, a partir das falhas e regularidades observadas, derivaram-se os requisitos do H2H v0.1.

A investigação ocorreu de forma intensiva ao longo de poucos dias (26 a 29 de julho de 2026) em uma máquina de desenvolvimento com Windows 11. Foram examinados oito harnesses, CLIs ou superfícies de controle: Claude Code, Codex CLI, Gemini CLI, Antigravity/`agy`, GitHub Copilot CLI, OpenCode, Cursor Agent CLI e Visual Studio Code. O VS Code foi usado como controle negativo conceitual, pois é uma superfície de edição e integração, mas não constitui, por si só, um harness delegado headless equivalente aos demais.

As versões das ferramentas variaram durante o período. Essa ausência de congelamento não permite comparação controlada entre builds, mas tornou visível um problema operacional relevante: a deriva de flags, comandos e capacidades ao longo das atualizações.

4.2 Unidade de análise

A unidade de análise foi uma execução delegada com objetivo verificável. Para cada execução, foram registrados, quando disponíveis:

- harness chamador e harness delegado;
- comando ou modo de invocação;
- diretório de trabalho;
- objetivo da tarefa;
- política de permissão solicitada;
- resultado observado;
- arquivos ou diffs produzidos;

- bloqueios de segurança;
- necessidade de interrupção humana;
- verificação posterior do artefato.

O conjunto principal reuniu seis execuções empíricas: uma prova sintética de geração de imagem e cinco tarefas de produção, duas capas de livro, um parecer editorial, uma correção de citações e a investigação de um defeito de software com três tentativas delegadas sucessivas.

4.3 Critério de sucesso

Uma execução não foi considerada bem-sucedida apenas porque o harness declarou ter concluído a tarefa. O resultado precisava ser confirmado por pelo menos um dos seguintes meios:

- inspeção visual direta do arquivo produzido;
- leitura integral do relatório gerado;
- conferência do diff;
- execução de testes ou verificadores do projeto;
- reexportação e validação da aplicação;
- confirmação do comportamento em hardware real.

4.4 Natureza do estudo

O estudo não mede qual harness é melhor. Os casos não foram equilibrados por modelo, versão, custo, número de tentativas ou complexidade. O objetivo é identificar padrões de composição, falhas e salvaguardas. Os resultados devem, portanto, ser interpretados como evidência de viabilidade e como geração de requisitos, não como ranking de produtos.

5. Proposta de protocolo H2H v0.1

5.1 Escopo

A versão 0.1 limita-se a delegação:

- local, na mesma máquina;
- síncrona do ponto de vista da tarefa raiz, ainda que o subprocesso seja monitorado de forma assíncrona;
- iniciada por subprocesso, CLI nativa ou sessão ACP local;
- com profundidade máxima igual a 1;
- entre um chamador e um delegado por subtarefa.

Ficam fora do escopo inicial: múltiplos saltos, delegação distribuída entre máquinas, ambiente multiusuário, consenso entre agentes, execução paralela em fan-out e arbitragem automática entre resultados conflitantes.

5.2 Manifesto de harness

Cada harness deve possuir um manifesto verificável contendo, no mínimo:

- identificador e versão detectada;
- fonte oficial de instalação;
- executável ou endpoint ACP;

- modo de invocação não interativa;
- parâmetro de diretório de trabalho;
- formatos de entrada e saída;
- códigos de retorno conhecidos;
- capacidades declaradas;
- níveis de autonomia suportados;
- dependências externas conhecidas;
- restrições de plataforma;
- comando de healthcheck;
- data da última validação do manifesto.

O manifesto não deve confiar em pacotes de terceiros com nomes semelhantes aos produtos oficiais sem verificação explícita de origem e mantenedor.

5.3 Descritor de tarefa

O descritor de tarefa deve separar objetivo, permissões e contexto. Um formato mínimo pode ser representado da seguinte maneira:

```
{
  "h2h_version": "0.1",
  "task_id": "uuid",
  "caller": "harness-a",
  "delegate": "harness-b",
  "objective": "descrição delimitada da subtarefa",
  "working_directory": "caminho absoluto",
  "target_artifacts": ["arquivo-ou-diretorio"],
  "autonomy_requested": 1,
  "timeout_seconds": 900,
  "output_format": "json",
  "previous_attempts": [],
  "delegation_depth": 1
}
```

O campo `previous_attempts` deve ser opcional e conter apenas informação útil: harness usado, hipótese, alteração aplicada e resultado observado. Esse histórico reduz a repetição e permite que um novo delegado procure o padrão comum entre tentativas anteriores.

5.4 Vocabulário comum de autonomia

Os harnesses usam nomenclaturas incompatíveis para permissões. O H2H propõe quatro níveis de referência:

Nível	Nome	Limite operacional
0	Consultivo	Leitura e análise, sem modificar artefatos e sem executar ações com efeitos persistentes.
1	Edição controlada	Pode modificar somente os artefatos-alvo ou o workspace designado; demais ações exigem aprovação.
2	Autonomia de workspace	Pode editar, executar comandos, testes e verificadores dentro do workspace, sem privilégios administrativos ou mudanças globais.

3	Autonomia ampliada	Pode solicitar ações mais amplas, inclusive rede ou dependências, mas continua sujeito às políticas e bloqueios do harness delegado.

O manifesto deve mapear esses níveis para as flags nativas disponíveis. Quando não houver equivalência exata, o delegado deve aplicar o nível inferior mais seguro e informar o rebaixamento.

5.5 Guarda contra recursão

A v0.1 estabelece duas regras obrigatórias:

1. Um harness não pode invocar a si mesmo dentro da mesma tarefa raiz.
2. Um harness que recebeu uma tarefa por H2H não pode realizar nova delegação H2H nessa mesma tarefa.

A profundidade máxima igual a 1 reduz ciclos, explosão de custo, perda de atribuição e repetição de efeitos externos. Trabalhos sobre loops agentivos mostram que fluxos sem limites efetivos podem amplificar chamadas, custo e efeitos colaterais (HOU et al., 2026).

5.6 Soberania de permissão do delegado

O chamador pode solicitar um nível de autonomia, mas não pode obrigar o delegado a exceder suas políticas. O nível efetivo é determinado pelo menor valor entre:

- autonomia solicitada pelo chamador;
- autonomia permitida pelo usuário ou ambiente;
- autonomia suportada pelo harness delegado;
- restrições adicionais do sandbox ou sistema operacional.

Autorizações em linguagem natural não devem substituir mecanismos formais de permissão. Quando uma flag perigosa é bloqueada pelo delegado, o H2H deve registrar o bloqueio e prosseguir em nível inferior ou encerrar a tarefa.

5.7 Timeout, saída e retomada

O valor `timeout_seconds` deve ser imposto pelo chamador. Não basta incluí-lo no prompt ou no descritor. Ao expirar o limite, o chamador deve:

1. solicitar encerramento gracioso, quando possível;
2. encerrar o processo após um período curto de tolerância;
3. capturar `stdout`, `stderr`, código de saída e estado final;
4. verificar se os artefatos-alvo foram modificados;
5. impedir que uma nova tentativa sobrescreva alterações parciais não avaliadas.

Em falhas, a saída não deve ser truncada no registro técnico. Uma interface resumida pode mostrar apenas o essencial ao usuário, mas o log completo precisa ser preservado para diagnóstico.

5.8 Uso de ACP como transporte

Quando o delegado oferecer servidor ACP, uma implementação H2H pode preferi-lo à captura ad hoc de texto da CLI. ACP fornece comunicação estruturada, streaming, sessões e pedidos de

permissão. Ainda assim, as regras de seleção do delegado, profundidade, timeout, soberania de permissão e integridade de artefatos continuam pertencendo à camada H2H.

6. Estudos de caso

6.1 Fechamento de lacuna categórica: geração de imagens

6.1.1 Prova sintética de capacidade

Na configuração testada, o Claude Code não expunha ferramenta nativa de geração de imagens. O harness chamador delegou ao Codex CLI a criação de uma maçã vermelha sobre fundo branco. O ambiente delegado selecionou uma capacidade de geração de imagem e devolveu um arquivo PNG coerente com a solicitação, confirmado por inspeção visual.

O valor desse caso não está na dificuldade da imagem. Ele demonstra uma lacuna categórica: o chamador não possuía a ferramenta necessária, enquanto o delegado possuía. A delegação evitou a tentativa de simular a capacidade ausente ou de substituir apenas o modelo dentro do harness chamador.

6.1.2 Capa do livro sobre RISC-V

O padrão foi aplicado a uma capa de livro em produção. A arte de fundo foi gerada sem texto embutido. Em seguida, um script determinístico em PowerShell, usando `System.Drawing`, aplicou título, subtítulo, frase de efeito e nome do autor com fontes do sistema.

O arquivo final possuía 1600 × 2560 pixels e metadados de 300 DPI. A composição híbrida eliminou a dependência do modelo de imagem para reproduzir acentos e diacríticos. A fidelidade textual tornou-se determinística, limitada apenas pela disponibilidade da fonte, pelos parâmetros tipográficos e pelo mecanismo de renderização.

Esse fluxo também produziu um artefato reexecutável: o script de composição. Assim, alterações de texto ou posição podem ser refeitas sem nova geração da imagem de fundo.

6.1.3 Capa do livro sobre edição genética por CRISPR

Em uma segunda capa, um wrapper de automação terminou sem produzir `capa.png` e sem apresentar a causa no resumo exibido. A invocação direta do delegado, com captura integral de `stdout` e `stderr`, revelou que uma etapa do pipeline tentava usar ImageMagick, ausente na máquina.

O delegado substituiu essa etapa por uma solução em PowerShell e `System.Drawing`, concluindo a capa. A observação não permite afirmar que toda a skill de imagem dependa de ImageMagick. O que foi comprovado é que o pipeline executado naquela tentativa possuía uma dependência externa não satisfeita e não declarada previamente.

Duas regras foram derivadas:

- wrappers não devem ocultar a saída completa quando a execução falhar;
- manifestos devem declarar executáveis externos usados por skills, scripts ou etapas geradas dinamicamente.

6.2 Delegação de julgamento: trabalho editorial e analítico

6.2.1 Parecer editorial estruturado

O Codex recebeu o papel de editor profissional para avaliar o rascunho de um livro. A instrução exigia que cada achado fosse classificado conforme a força de intervenção necessária.

O resultado foi um parecer de aproximadamente 23 KB, lido integralmente, com resumo executivo, pontos fortes e 17 problemas. Quatro foram classificados como `CORREÇÃO SIMPLES` e treze como `MUDANÇA ESTRUTURAL/EDITORIAL FORTE`.

O principal resultado não foi apenas o conteúdo do parecer, mas a estrutura de decisão. A autoclassificação por impacto permitiu separar intervenções potencialmente automatizáveis de decisões que deveriam permanecer com o autor. Esse padrão transfere para a saída analítica a mesma lógica usada nos níveis de autonomia operacional.

6.2.2 Correção de citações com regra antifabricação

Em outro manuscrito, doze nomes apareciam com citações inconsistentes ou sem entrada bibliográfica suficiente. A tarefa delegada estabeleceu uma regra conservadora:

- usar o sistema autor-data com os anos já sustentados pela bibliografia;
- nunca inventar número de página;
- pesquisar fonte real quando faltasse referência;
- remover a atribuição nominal e generalizar a afirmação quando não houvesse confirmação segura.

A verificação ocorreu por diff e por ferramentas do projeto. Foram revisadas 27 citações; 22 foram classificadas como fortes pelo verificador interno; nenhuma permaneceu como não localizada; e a análise final não registrou problema crítico. O delegado localizou fontes com DOI para dois autores, removeu dois nomes sem suporte seguro e identificou uma atribuição incorreta adicional.

O caso sugere que restrições explícitas, conservadoras e verificáveis podem ser delegadas com maior segurança do que instruções genéricas como “corrija as referências”. A regra relevante não foi “encontre uma resposta”, mas “não fabrique quando a evidência for insuficiente”.

6.3 Triangulação de um defeito real

6.3.1 Problema observado

Um protetor de tela 3D desenvolvido em Godot funcionava no monitor primário, mas apresentava tela preta no monitor secundário.

6.3.2 Primeira tentativa: Codex CLI

O Codex leu o código e consultou documentação sobre múltiplas janelas. A execução, porém, permaneceu por mais de quinze minutos sem progresso perceptível. Suspeitou-se de bloqueio em uma etapa de rede dentro do sandbox, mas essa causa não foi confirmada.

Como o chamador não impunha timeout automático, a interrupção dependeu de decisão humana. Antes de retomar o trabalho, verificou-se que nenhum arquivo havia sido alterado. O caso demonstrou que timeout documentado, mas não imposto, não é controle efetivo.

6.3.3 Segunda tentativa: Antigravity/`agy`

O segundo delegado propôs que câmeras e texturas ligadas diretamente a uma `Window` nativa criada em tempo de execução poderiam não possuir pipeline de desenho confiável no Godot 4. A recomendação foi adotar `SubViewportContainer` e `SubViewport` com `world_3d` compartilhado.

A alteração foi implementada, validada quanto a erros e reexportada. O defeito persistiu. A hipótese era tecnicamente plausível, mas não explicava o comportamento observado no hardware.

6.3.4 Terceira tentativa: GitHub Copilot CLI

A terceira delegação incluiu o histórico das tentativas anteriores. O Copilot observou que ambas preservavam um elemento comum: a criação de uma segunda janela nativa do Windows. A hipótese proposta foi uma falha na criação ou apresentação de uma segunda swapchain Vulkan quando os monitores estavam conectados a saídas gráficas diferentes.

A solução aplicada reuniu a renderização em uma única janela, com uma região `SubViewport` para cada monitor. O projeto foi reexportado e o usuário confirmou o funcionamento no hardware real.

O sucesso da correção sustenta a hipótese da segunda swapchain, mas não a prova conclusivamente, pois não foram coletados logs de driver ou rastreamento Vulkan. A formulação correta é: o terceiro diagnóstico produziu a explicação operacional mais consistente e a única correção confirmada no ambiente real.

6.3.5 Lições derivadas

O caso sustenta quatro requisitos:

1. consultar diferentes harnesses pode revelar hipóteses complementares;
2. o histórico de tentativas deve acompanhar novas delegações;
3. timeout precisa ser aplicado pelo chamador;
4. o estado dos artefatos deve ser verificado antes da retomada.

Também surgiu uma lição de supervisão. Durante tarefas longas, foram criados monitores duplicados por impaciência, embora o primeiro continuasse ativo. A regra operacional derivada é simples: uma tarefa deve possuir um único monitor responsável, identificado por `task_id`.

6.4 Achados transversais

Autonomia sem atrito. Na matriz observada, a combinação Claude Code como chamador e GitHub Copilot CLI como delegado foi a única que aceitou o nível de autonomia solicitado sem bloqueio adicional. Esse resultado é específico às versões e configurações testadas e não deve ser generalizado como propriedade permanente dos produtos.

Bloqueio compatível com proteção contra recursão. A tentativa de usar Codex CLI para invocar Claude Code foi impedida por uma proteção consistente com bloqueio de sessão aninhada ou recursiva. O resultado reforçou a adoção da profundidade máxima igual a 1.

Risco de cadeia de suprimentos. Em testes relacionados a Antigravity e Cursor Agent CLI, foram encontrados pacotes ou repositórios de terceiros com nomes semelhantes aos produtos oficiais. Nenhum foi instalado. O manifesto H2H deve registrar somente fontes oficiais ou fontes cuja procedência tenha sido verificada.

Deriva de manifesto. Uma atualização do Codex CLI removeu uma flag usada no comando documentado de delegação. Nenhum aviso foi percebido no fluxo local, e a falha somente foi identificada durante nova execução. Esse caso motivou healthchecks que comparem a saída atual de `--help` com o manifesto conhecido.

7. Discussão

7.1 Três modos de valor

Os casos apontam três modos de valor independentes:

1. **Capacidade:** o delegado possui uma ferramenta ou integração ausente no chamador.
2. **Julgamento:** o delegado oferece uma leitura independente, uma especialidade ou uma crítica estruturada.
3. **Triangulação:** diferentes harnesses formulam hipóteses distintas para um problema que resiste a uma única linha de investigação.

Essa classificação evita reduzir o H2H a simples “troca de modelo”. No primeiro modo, a vantagem pode estar em uma ferramenta nativa. No segundo, está na independência do julgamento. No terceiro, está na diversidade de caminhos diagnósticos e na capacidade de incorporar o histórico das tentativas.

7.2 Relação com protocolos existentes

MCP pode expor ferramentas especializadas ao chamador. A2A pode conectar agentes independentes por serviços de rede. ACP pode fornecer transporte estruturado entre cliente e agente. Planos de controle podem coordenar frotas de agentes e execuções paralelas.

O H2H ocupa uma camada menor: define como um harness local decide e controla a invocação de outro harness completo, especialmente quando o único ponto comum disponível é uma CLI headless. Sua contribuição não está em criar um novo formato universal de mensagens, mas em formalizar requisitos operacionais que podem ser aplicados sobre CLI nativa ou ACP.

7.3 Segurança e responsabilidade

A delegação não transfere integralmente a responsabilidade. O chamador continua responsável por:

- escolher o delegado;
- limitar o objetivo;
- solicitar autonomia proporcional;
- impor timeout;
- preservar logs;
- validar artefatos;
- impedir recursão;
- decidir se o resultado será incorporado.

O delegado permanece soberano sobre suas próprias políticas de permissão. Essa divisão evita dois extremos: um chamador incapaz de controlar os efeitos do subprocesso e um chamador capaz de desativar unilateralmente todas as proteções do delegado.

7.4 Reprodutibilidade prática

A principal ameaça à reprodutibilidade é a rápida evolução das CLIs. Comandos, flags, modelos e autenticação mudam com frequência. Por isso, reproduzir H2H exige mais do que registrar o prompt. É necessário registrar versão do executável, saída de `--help`, sistema operacional, diretório de trabalho, configuração de permissões, dependências externas e códigos de saída.

O manifesto deve ser tratado como artefato versionado e testável, não como documentação estática.

8. Limitações e ameaças à validade

Este estudo possui limitações relevantes:

- foi conduzido pelos autores, em uma única máquina e em um único sistema operacional;
- o período de observação foi curto e intensivo (poucos dias), não longitudinal; isso favoreceu a coleta concentrada de casos de produção reais, mas não permite avaliar estabilidade dos harnesses, recorrência de falhas ou drift de manifesto ao longo do tempo, apenas o que foi observado na janela estudada;
- mesmo nessa janela curta, as versões dos harnesses não permaneceram congeladas (foram observadas ao menos duas atualizações de CLI em poucos dias), o que já sinaliza que a instabilidade de interface tende a ser mais severa, não menos, em janelas mais longas;
- as tarefas não foram distribuídas de forma balanceada entre todos os produtos;
- parte dos sistemas é proprietária e opaca, o que limita a inspeção da causa interna das falhas;
- os casos de produção pertencem ao fluxo de trabalho dos autores e podem não representar outros domínios;
- não houve replicação independente;
- algumas causas foram inferidas a partir do comportamento e da correção funcional, sem telemetria de baixo nível;
- o estudo não controla diferenças de modelo, contexto, custo, latência ou número de tentativas.

Essas limitações impedem conclusões comparativas sobre qualidade geral dos harnesses. Elas não eliminam, contudo, a utilidade dos casos para revelar classes de falha e gerar requisitos de protocolo. A validação futura deve incluir replicação em Linux e macOS, versões congeladas, matriz de delegações reproduzível e publicação dos logs e artefatos sanitizados.

9. Trabalhos futuros

A evolução para H2H v0.2 deve priorizar:

1. implementação de referência com timeout real e encerramento seguro;
2. esquema JSON validável para manifestos e descritores de tarefa;
3. healthcheck automático para detectar deriva de CLI;
4. suporte a histórico de tentativas e correlação por `task_id`;
5. registro centralizado de custo, tokens, duração e artefatos;
6. adaptadores ACP para harnesses compatíveis e fallback para CLI nativa;
7. política de roteamento por capacidade, risco e orçamento;
8. conjunto público de testes com tarefas sintéticas e casos de produção sanitizados;

9. estudo posterior de fan-out e profundidade superior a 1, somente após mecanismos formais de prevenção de ciclos.

10. Conclusão

Este artigo definiu H2H como um padrão de delegação local em que um harness de agente de codificação invoca outro harness completo como subprocesso, sem depender de um orquestrador externo. A proposta nasceu de um período curto e intensivo de observação empírica (poucos dias) e de seis execuções principais, cinco em produção e uma sintética.

Os casos demonstraram três formas de valor: fechar lacunas de capacidade, obter julgamento independente e triangular diagnósticos. Também mostraram que a composição entre harnesses introduz riscos próprios, especialmente recursão, autonomia mal mapeada, timeout não imposto, perda de diagnóstico, dependências externas não declaradas e deriva de interface.

O H2H v0.1 responde com um núcleo deliberadamente pequeno: manifesto de harness, descritor de tarefa, quatro níveis de autonomia, profundidade máxima igual a 1, soberania de permissão do delegado, captura integral de falhas e verificação de artefatos antes da retomada.

A principal conclusão é que harnesses não precisam ser tratados como ambientes isolados nem reduzidos aos modelos que utilizam. Eles podem tornar-se unidades composicionais. Para que essa composição seja confiável, porém, a delegação precisa deixar de ser um script improvisado e passar a possuir contrato, limites, observabilidade e responsabilidade explícita.

Referências

A2A PROJECT. **Agent2Agent Protocol Specification**. Linux Foundation, 2026. Disponível em: https://a2aproject.github.io/A2A/latest/specification/. Acesso em: 29 jul. 2026.

AGENT CLIENT PROTOCOL. **Architecture**. 2026a. Disponível em: https://agentclientprotocol.com/get-started/architecture. Acesso em: 29 jul. 2026.

AGENT CLIENT PROTOCOL. **Introduction**. 2026b. Disponível em: https://agentclientprotocol.com/get-started/introduction. Acesso em: 29 jul. 2026.

ANTHROPIC. **Claude Code CLI reference**. 2026. Disponível em: https://code.claude.com/docs/en/cli-usage. Acesso em: 29 jul. 2026.

CHEN, Tingyang; LU, Shuo; ZHAO, Kang; MENG, Weicheng; TENG, Hanlin; LI, Tianhao; LI, Chao; LIU, Xule; LIANG, Jian; ZHANG, Zhizhong; XIE, Yuan; QU, Heng; SHAO, Kun; LUAN, Jian. **HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2606.14249. Acesso em: 29 jul. 2026.

GITHUB. **GitHub Copilot CLI programmatic reference**. 2026. Disponível em: https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-programmatic-reference. Acesso em: 29 jul. 2026.

GOOGLE. **Gemini CLI: headless mode reference**. 2026. Disponível em: https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/headless.md. Acesso em: 29 jul. 2026.

HOU, Xinyi; WANG, Shenao; ZHAO, Yanjie; WANG, Haoyu. **When Agents Do Not Stop: Uncovering Infinite Agentic Loops in LLM Agents**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2607.01641. Acesso em: 29 jul. 2026.

MODEL CONTEXT PROTOCOL. **Architecture** (Protocol revision 2025-06-18). 2025. Disponível em: https://modelcontextprotocol.io/specification/2025-06-18/architecture. Acesso em: 29 jul. 2026.

PAN, Linyue; ZOU, Lexiao; GUO, Shuo; NI, Jingchen; ZHENG, Hai-Tao. **Natural-Language Agent Harnesses**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2603.25723. Acesso em: 29 jul. 2026.

SAYED ALI, Muayad; NOVIK, Aliaksandra; BODDUPALLY, Anji; YAVORSKYI, Artem; NICKERSON, Chris; RICA, Daniel; DUGRANRUT, Emily; LEUNG, Felix; PRINCE, Garrett; BARNETT, Grace; ROBINSON, Heath; AL AHMAD, Hosain; RESNICK, Jesse; FARAH, Juan Carlos; MERUGA, Jyothi Swaroop; KUZNETSOV, Leonid; PERRY, Brock; GORHAM, Luke; SCHMOLL, Marie *et al.* **The Harness Effect: How Orchestration Design Sets the Token Economics of Enterprise Agentic AI**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2607.06906. Acesso em: 29 jul. 2026.

WANG, Ruhan; SHI, Yucheng; LI, Zongxia; LI, Zhongzhi; YU, Yue; YANG, Junyao; PANAGANTI, Kishan; MI, Haitao; ZHOU, Dongruo; LEOWEILIANG. **Harness Handbook: Making Evolving Agent Harnesses Readable, Navigable, and Editable**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2607.13285. Acesso em: 29 jul. 2026.

WARP. **A single pane of glass for managing all of your cloud agents**. 19 maio 2026. Disponível em: https://www.warp.dev/blog/multi-harness-cloud-agent-orchestration. Acesso em: 29 jul. 2026.

ZHONG, Hailin; ZHU, Shengxin. **AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents**. arXiv, 2026. Disponível em: https://arxiv.org/abs/2605.13357. Acesso em: 29 jul. 2026.